

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)
Department of Computer Science and Engineering

ASSESSMENT TOOL 1 – ANALYTICAL PROBLEM SOLVING

Course Code:	CSA0305	Course Title:	Data Structures
CO Assessed:	CO2 – Manipulation (BL3, BL4)	Assessment:	Assessment Tool 1 – Analytical Problem Solving
Weightage:	30% of CIA	Total Marks:	100
CO2 Statement:	Implement linear data structures such as stacks, queues, and linked lists, and analyze the efficiency of linear and binary search techniques.		
Student Name:	RAGUL.M	Reg. No.:	192524066
Section / Batch:	SLOT-A / CSA0305	Date:	20/07/2026

Code of Conduct

I RAGUL.M / 192524066 (Name / Reg No)

certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: Ragul.M

Instructions

- This test contains 5 Analytical problem-solving questions. Total: 100 Marks.
- When a student answer using an Analytical problem-solving, in evaluation the answer should be with following five requirements: Understanding of problem, select appropriate data structure, Design the solution, analyze, Clarity of representation.
- Each student should write 2 questions. No negative marking. Unanswered questions carry zero marks.
- No DTP printouts or electronic devices permitted. They should provide written materials.

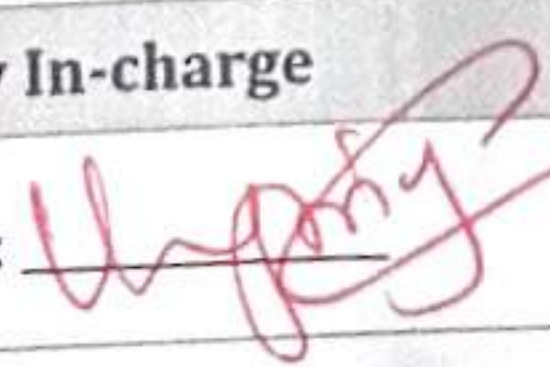
Section / Topic	Focus	Q Nos	Marks
Linked Data Structure, Search Data Structure	List Operations, Binary Search	1	50
Stack Data Structure, Queue Data Structures	Stack Notations, Priority Queue	2	50
GRAND TOTAL:		100 Marks	

26	192524066	RAGUL M	- Identify if the given expressions are balanced or not. (i) $[9 * (k+8 - \{ 6 + e \} + 77) / 100]$, (ii) $((a*b) + [u/2] - d)$. - Given a stack with repeated elements, design an approach to compress it (value & frequency) while maintaining stack operations. Apply stack ADT modifications. Analyze trade-off between time and space
27	192511260	S DHARSHINI	- A circular queue has a size of 6. Execute the following operations: Enqueue(10), Enqueue(20), Enqueue(30), Dequeue(), Enqueue(40), Enqueue(50), Enqueue(60), Enqueue(70). Draw the queue after each operation and identify the values of the front and rear pointers. - Design a system that dynamically chooses between linear search and binary search based on: ➤ Data size ➤ Sortedness ➤ Frequency of queries Analyze decision criteria mathematically
28	192471038	SASI KUMAR MEGHA	- Insert the following elements 100, 200, 300, 400, 500 into the linked stack and pop the elements 100 and 200 from the linked stack. Show the linked structure before and after linked stack operations. - An online food delivery service receives the following orders: O1, O2, O3, O4. While O2 is being prepared, an express order O5 arrives. <i>Questions:</i> ➤ Explain why a normal queue may not satisfy customer requirements. ➤ Recommend an appropriate queue implementation and justify your choice.
29	192424359	SUNDARESAN R U	- Construct an efficient system using two stacks within a single array structure to maximize memory utilization. Compare fixed partitioning and dynamic allocation strategies, and analyze how each approach affects overflow conditions, space efficiency, and operational complexity under varying workloads. - A queue-based ticketing system serves customers in the order of arrival, but VIP customers must be prioritized occasionally. Analyze how modifying a simple queue into a priority-based queue affects fairness, waiting time, and computational complexity. Compare both systems using analytical reasoning.
30	192524479	SHAHANA	- Convert the following infix expression into postfix notation. Show the stack contents after processing each symbol. $((A+B)*C-(D-E)*(F+G))/H$ - For the expression. $((8+4)*(7-3)/(2+1))+(5*(6-2))$ Perform the following:

38	192524251	VAISHNAV M	- Two algorithms solve the same problem. Algorithm A uses recursion, while Algorithm B uses an explicit stack. Compare both approaches in terms of memory usage, execution flow, and risk of stack overflow. - A stack has a capacity of 5 elements. Explain what happens if the following operations are executed: Push(A), Push(B), Push(C), Push(D), Push(E), Push(F). Identify the error and suggest two possible solutions.
39	192525187	VEERA SANTHOSH A	- Compare the performance of a linear queue and a circular queue when 100 enqueue and dequeue operations are performed alternately. - A queue implemented using an array report "Queue Full" even though some positions are empty. - Analyze why this occurs. - Explain how a circular queue solves this problem.
40	192511226	VINIL KUMAR S	- Consider the singly linked list: $10 \rightarrow 20 \rightarrow 30 \rightarrow 40 \rightarrow 50$. Delete the node containing 30. Explain how the links change after deletion. - Given two sorted linked lists: L1: $1 \rightarrow 3 \rightarrow 5 \rightarrow 7$ L2: $2 \rightarrow 4 \rightarrow 6 \rightarrow 8$

Rubrics for Calculation

Criteria	Weightage	Level 4 - Exemplary	Level 3 - Proficient	Level 2 - Developing	Level 1 - Beginning
Problem Understanding	20	Demonstrates complete and precise understanding; identifies all constraints and edge cases	Shows clear understanding with minor gaps	Partial understanding; misses some constraints	Misinterprets problem or lacks clarity
Data Structure Selection	20	Selects optimal data structure with strong justification and comparison	Selects appropriate structure with reasonable justification	Selects somewhat suitable structure with weak reasoning	Incorrect or no data structure selection
Algorithm Design	20	Designs efficient, logically sound, and well-structured algorithm	Algorithm is correct with minor inefficiencies	Algorithm is partially correct or lacks clarity	Algorithm is incorrect or missing
Accuracy of Solution	30	Error-free, well-structured, optimized code/solution	Minor errors but overall functional	Multiple errors; partially working	Non-functional or missing solution
Clarity	10	Exceptionally clear, well-organized, uses diagrams effectively	Clear and organized with minor issues	Somewhat organized but lacks clarity	Poorly organized and unclear

Faculty In-charge	Course Coordinator	Program Director
Signature: 	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

NAME: RAGINI.M

REG: 192524066

DATE: 10.08.2026

Subject: Data Structures

CODE: CSA0305

Assessment: AT1

1. Identify whether the given expressions are balanced or not.

1. Problem Identification

An expression is said to be balanced if every opening bracket has a corresponding closing bracket in the correct order.

The brackets are

⇒ () - Parentheses

⇒ [] - Square brackets

⇒ { } - Curly brackets

A stack is suitable for this problem because it follows LIFO (Last in First out).

2 problem - Solving Approach.

We Scan the expression from left to right

Rules:

1. $\Rightarrow$ if an opening bracket ${, [, {$ is found $\rightarrow$ push in stack

2. $\Rightarrow$ if a closing bracket $),], }$ is found:

$\Rightarrow$ check the top of the stack

$\Rightarrow$ if it is the matching opening bracket

$\rightarrow$ pop

$\Rightarrow$ otherwise $\rightarrow$ not Balanced

3. After Scanning the complete expression

$\Rightarrow$ stack empty - Balanced

$\Rightarrow$ stack not empty - not Balanced.

3 Algorithm

1. create an empty stack S .

2. scan each character of expression from left to right

3. if character is ${, [, {$:

push character into S .

4. if character is $),], }$ or $'$:

If S is empty:

Return "not Balanced"

$x = \text{POP}(S)$

if x and current character are not matching:

Return "not Balanced"

$x = \text{POD}(S)$

if x and current character are not matching,
return "not Balanced"

5. After Scanning all characters:

if S is empty:

Return "Balanced"

Else

Return "not Balanced"

Expression (i)

Given

$[9 * (x + 8 - \{6 + e\} + 77) / 100]$

Stack Trace

Symbol	operation	stack
[	push	[
(	push	[(
{	push	[({
}	pop	[(
)	pop	[

Stack = [

Result

not Balanced.

Reason:

one opening square bracket [remains unmatched.

Expression (ii)

$$c((a * b) + [u/2] - d)$$

Stack Trace

Symbol	operation	Stack
(	Push	(
(	Push	((
(	Push	(((
)	pop	((
[	push	((([
]	pop	((
)	POP	(

Stack = (

Result

not Balanced.

Reason:

There one unmatched opening parentheses.

7. complexity Analysis.

Let n be the number of characters in the expression

Time complexity

Each character is scanned only once

$$\text{Time} = O(n)$$

Space complexity.

In the worst case, all characters can be opening brackets and stored in the stack

$$\text{Space} = O(n)$$

8. Conclusion.

The stack ADT provides an efficient solution for checking balanced expressions because the most recently encountered opening bracket must be matched first. Both given expressions contain unmatched opening brackets, so both expressions are not balanced.

2

compress a stack with repeated elements.

1. Problem identification

In a normal stack, repeated elements are stored separately.

Example

A A A B B C C C

normal stack stores 8 elements. To reduce memory, store elements as (value, frequency)

(C, 3)

(B, 2)

(A, 3)

This is called as stack compression

2. modified stack structure.

Each stack node contains:

(value, frequency)

Ex

(A, 3) means A occurs 3 times

3. Push operation

1. if stack is empty $\rightarrow$ insert (x, 1)

2. if top value equals x $\rightarrow$ increase frequency.

3. otherwise $\rightarrow$ create a new (x, 1) node.

Ex:

Push(A) $\rightarrow$ (A,1)

Push(A) $\rightarrow$ (A,2)

Push(A) $\rightarrow$ (A,3)

Push(B) $\rightarrow$ (B,1), (A,3)

Time Complexity: $O(1)$

4 POP operation

when POP() is performed:

1. if stack is empty $\rightarrow$ underflow
2. if top frequency $> 1 \rightarrow$ decrease frequency by 1
3. if frequency = 1 $\rightarrow$ Remove the complete node
4. Return the popped value.

Ex:

Before: (A,3)

POP()

After: (A,2)

Time Complexity: $O(1)$

6 Example

Input:

AAA BBCCC

Normal stack

C
C
C
B
B
A
A
A

8 entries

Compressed Stack

Top

(C, 3)

(B, 2)

(A, 3)

Only 3 entries are required

7 Stack ADT operations.

The modified stack ADT supports.

PUSH (x)

POP()

PEEK()

isEmpty()

All basic operations continue to work according to LIFO

8 Time-Space Trade-off

Space:

$\Rightarrow$ Normal Stack $\rightarrow O(n)$

$\Rightarrow$ Compressed Stack $\rightarrow O(k)$

where

$\Rightarrow n = \text{total number of elements}$

$\Rightarrow k = \text{number of different consecutive groups.}$

9 Advantages

$\Rightarrow$ Reduces memory usage

$\Rightarrow$ maintain normal stack behavior

$\Rightarrow$ PUSH and POP remain $O(1)$

$\Rightarrow$ Frequency information is directly available

Limitation

$\Rightarrow$ if elements are not repeated.

consequently compression provides very little space saving

10 Conclusion

The value-frequency approach compresses repeated stack elements efficiently. It maintains the stack ADT and LIFO behavior, while reducing space from $O(n)$ to $O(k)$ when repeated elements are present.